

Module #3 Introduction to OOPS Programming

1. Introduction to C++

THEORY EXERCISE:

1. What are the key differences between Procedural Programming and Object-Oriented Programming (OOP)?

Answer:

Procedural Programming (POP)	Object-Oriented Programming (OOP)
Focuses on functions and procedures.	Focuses on objects and classes.
Data and functions are separate.	Data and functions are combined into objects.
Follows a top-down approach.	Follows a bottom-up approach.
Less secure because data can be accessed easily.	More secure through data hiding and encapsulation.
Suitable for small programs.	Suitable for large and complex programs.
Examples: C, Pascal.	Examples: C++, Java, Python.

2. List and explain the main advantages of OOP over POP.

Answer:

Encapsulation (Data Security)

- OOP combines data and methods into a single unit called an object.
- Data can be protected from unauthorized access.

Code Reusability

- Through inheritance, existing classes can be reused to create new classes.
- This reduces code duplication.

Modularity

- Programs are divided into smaller classes and objects.
- Makes code easier to manage and understand.

Easy Maintenance

- Changes made in one class usually do not affect the entire program.
- Easier to update and debug large applications.

Polymorphism

- The same function can perform different tasks depending on the object.
- Improves flexibility and extensibility.

Abstraction

- Hides unnecessary implementation details from the user.
- Allows users to focus on essential features only.

Better Real-World Modelling

- Objects represent real-world entities such as students, cars, and employees.
- Makes program design more intuitive.

3. Explain the steps involved in setting up a C++ development environment.

Answer:

Steps:

1. Install a C++ Compiler

A compiler converts C++ code into an executable program.

Examples:

- GCC (MinGW)
- Clang
- MSVC (Microsoft Visual C++)

2. Install an IDE

An IDE (Integrated Development Environment) helps write, compile, and run programs easily.

Examples:

- Dev-C++
- Code::Blocks
- Visual Studio
- VS Code

3. Configure the Compiler

- Open the IDE.

- Select the installed compiler.
- Set the compiler path if needed.

4. Create a New C++ File

- Click **File** → **New**.
- Create a new source file.
- Save it with a **.cpp** extension.

5. Write the Program

Enter your C++ code in the editor.

6. Compile the Program

Click **Compile** or **Build** to check for errors and create an executable file.

7. Run the Program

Click **Run** or **Compile & Run** to execute the program.

8. Check the Output

View the output in the console window and verify the result.

4. What are the main input/output operations in C++? Provide examples.

Answer:

i. **Cin:**

- cin is used to take input from the user.

Example:

```
#include <iostream>
using namespace std;

int main() {
    int age;
```

```
cout << "Enter your age: ";  
cin >> age;
```

```
return 0;  
}
```

ii. cout:

- cout is used to display output on the screen.

Example:

```
#include <iostream>  
using namespace std;  
  
int main() {  
    cout << "Hello, World!";  
    return 0;  
}
```

2. Variables, Data Types, and Operators

THEORY EXERCISE:

1. What are the different data types available in C++? Explain with examples.

Answer:

Data Type	Description	Example
int	Stores whole numbers	int age = 20;
float	Stores decimal numbers	float height = 5.8f;
double	Stores large decimal numbers	double pi = 3.14159;
char	Stores a single character	char grade = 'A';
bool	Stores true or false	bool pass = true;
string	Stores text	string name = "Keyur";
void	Represents no value	void display();

2. Explain the difference between implicit and explicit type conversion in C++.

Answer:

1. Implicit Type Conversion

- Done automatically by the compiler.
- Converts a smaller data type to a larger data type.

2. Explicit Type Conversion (Type Casting)

- Done manually by the programmer.
- Used when you want to force a conversion.

3. What are the different types of operators in C++? Provide examples of each.

Answer:

Operator	Example
Arithmetic Operators (+, -, *, /, %)	a + b
Relational Operators (==, !=, >, <, >=, <=)	a > b
Logical Operators (&&, `	
Assignment Operators (=, +=, -=, *=, /=)	a += 5
Bitwise Operators (&, `	, ^, <<, >>`)
Increment/Decrement Operators (++ , --)	a++, b--

4. Explain the purpose and use of constants and literals in C++.

Answer:

Constants

- A **constant** is a value that cannot be changed during program execution.

- **Example:**

```
const float PI = 3.14;
```

Literals

- A **literal** is a fixed value written directly in the program.

- **Examples:**

```
10    // Integer literal
3.14   // Floating-point literal
'A'    // Character literal
"Hello" // String literal
true   // Boolean literal
```

3. Control Flow Statements

THEORY EXERCISE:

1. What are conditional statements in C++? Explain the if-else and switch statements.

Answer:

- **Conditional statements** are used to make decisions in a program based on certain conditions.

1. if-else Statement

The if-else statement executes different blocks of code depending on whether a condition is true or false.

Example:

```
int age = 18;

if (age >= 18)
    cout << "Eligible to vote";
else
    cout << "Not eligible to vote";
```

2. switch Statement

- The switch statement is used to select one block of code from multiple options.

Example:

```
int day = 2;

switch(day) {
    case 1:
        cout << "Monday";
        break;
    case 2:
        cout << "Tuesday";
        break;
    default:
        cout << "Invalid Day";
}
```

2. What is the difference between for, while, and do-while loops in C++?

Answer:

Loop	Description
------	-------------

for	Used when the number of iterations is known.
while	Used when the number of iterations is not known. Checks the condition before execution.
do-while	Executes the loop body at least once, then checks the condition.

3. How are break and continue statements used in loops? Provide examples.\

Answer:

1. break Statement

- The break statement immediately terminates the loop.

Example:

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 5; i++) {
        if (i == 3)
            break;
        cout << i << " ";
    }
    return 0;
}
```

2. continue Statement

- The continue statement skips the current iteration and moves to the next iteration of the loop.

Example:

```
#include <iostream>
using namespace std;

int main() {
```

```

    for (int i = 1; i <= 5; i++) {
        if (i == 3)
            continue;
        cout << i << " ";
    }
    return 0;
}

```

4. Explain nested control structures with an example

Answer:

- **Nested control structures** are control statements placed inside another control statement. The most common example is a **loop inside another loop** (nested loop).

Example:

```

#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 5; i++) {
        for (int j = 1; j <= i; j++) {
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}

```

4. Functions and scope

THEORY EXERCISE:

1. What is a function in C++? Explain the concept of function declaration, definition, and calling.

Answer:

- A **function** is a block of code that performs a specific task. Functions help make programs organized, reusable, and easier to understand.

1. Function Declaration

- A function declaration tells the compiler about the function's name, return type, and parameters.
- `int add(int, int);`

2. Function Definition

- A function definition contains the actual code that performs the task.
- ```
int add(int a, int b) {
 return a + b;
}
```

## 3. Function Calling

- A function is called when you want to execute it.
- ```
int result = add(10, 20);
```

2. What is the scope of variables in C++? Differentiate between local and global scope.

Answer:

Local Scope

- A **local variable** is declared inside a function or block and can only be accessed within that function or block.

Example:

```
void display() {  
    int x = 10; // Local variable  
    cout << x;  
}
```

Global Scope

- A **global variable** is declared outside all functions and can be accessed by any function in the program.

Example:

```
int x = 10; // Global variable
```

```
void display() {  
    cout << x;  
}
```

Difference Between Local and Global Scope

Local Scope	Global Scope
Declared inside a function or block	Declared outside all functions
Accessible only within that function/block	Accessible throughout the program
Exists only during function execution	Exists for the entire program execution

3. Explain recursion in C++ with an example.

Answer:

- **Recursion** is a technique where a function calls itself to solve a problem. A recursive function must have a **base case** to stop the recursion.

Example:

```
#include <iostream>
using namespace std;

int factorial(int n) {
    if (n == 0 || n == 1)
        return 1;    // Base case
    else
        return n * factorial(n - 1);
}

int main() {
    int num = 5;

    cout << "Factorial = " << factorial(num);

    return 0;
}
```

4. What are function prototypes in C++? Why are they used?

Answer:

A function prototype is a declaration of a function that tells the compiler:

- The function's name
- The return type
- The number and types of parameters

Why are Function Prototypes Used?

1. Inform the Compiler
 - The compiler knows about the function before it is called.
2. Enable Function Calls Before Definition
 - A function can be called in main() even if its definition appears later in the program.
3. Type Checking

- The compiler checks whether the correct number and type of arguments are passed.
4. Improve Program Organization
- Functions can be defined separately from their usage, making code easier to read and maintain.

5. Arrays and Strings

THEORY EXERCISE:

3. What are arrays in C++? Explain the difference between single-dimensional and multidimensional arrays.

Answer:

- An **array** is a collection of elements of the **same data type** stored in **contiguous memory locations**. Arrays allow you to store multiple values using a single variable name.

Feature	Single-Dimensional Array	Multidimensional Array
Definition	Stores data in a single row or list.	Stores data in rows and columns (table format).
Dimensions	One dimension.	Two or more dimensions.
Declaration	<code>int arr[5];</code>	<code>int arr[2][3];</code>
Access	One index is used.	Multiple indices are used.
Example Use	Storing marks of students.	Storing a matrix or table of data.

2. Explain string handling in C++ with examples.

Answer:

- A **string** is a sequence of characters used to store and manipulate text such as names, messages, and sentences. In C++, strings are provided by the **string** class from the `<string>` library.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string name = "Keyur";
    cout << name;
    return 0;
}
```

3. How are arrays initialized in C++? Provide examples of both 1D and 2D arrays.

Answer:

- **Array initialization** is the process of assigning values to array elements when the array is declared.

Syntax:

1D Array:

```
data_type array_name[size] = {values};
```

2D Array:

```
data_type array_name[rows][columns] = {{values}, {values}};
```

1. One-Dimensional (1D) Array Initialization

A 1D array stores elements in a single row.

```
#include <iostream>
using namespace std;

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};

    for (int i = 0; i < 5; i++) {
        cout << numbers[i] << " ";
    }

    return 0;
}
```

2. Two-Dimensional (2D) Array Initialization

A 2D array stores data in rows and columns (matrix form).

```
#include <iostream>
using namespace std;

int main() {
    int matrix[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

4. Explain string operations and functions in C++.

Answer:

- A **string** is a sequence of characters used to store text. In C++, strings are handled using the string class from the <string> library.

```
#include <string>
using namespace std;
```

Common String Operations

1. String Initialization

```
string str1 = "Hello";
string str2("World");
```

2. String Input and Output

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string name;

    cout << "Enter your name: ";
    getline(cin, name);

    cout << "Hello, " << name;
    return 0;
}
```

Function	Description
length()	Returns the length of the string
size()	Returns the size of the string
append()	Adds another string
substr()	Extracts a substring
find()	Finds a character or substring
erase()	Removes characters
empty()	Checks if the string is empty
clear()	Removes all characters
compare()	Compares two strings

6. Introduction to Object-Oriented Programming

THEORY EXERCISE:

1. Explain the key concepts of Object-Oriented Programming (OOP).

Answer:

1. Class

A **class** is a blueprint or template for creating objects. It defines the data members and member functions that an object will have.

Example

```
class Student {  
    public:  
        string name;  
        int age;  
};
```

Here, Student is a class.

2. Object

An **object** is an instance of a class. It is used to access the data members and functions of the class.

Example

```
Student s1;  
s1.name = "Rahul";  
s1.age = 20;
```

Here, s1 is an object of the Student class.

3. Encapsulation

Encapsulation is the process of wrapping data and functions into a single unit (class) and restricting direct access to data.

Example

```
class BankAccount {  
    private:  
        double balance;  
  
    public:  
        void deposit(double amount) {  
            balance += amount;  
        }  
};
```

4. Inheritance

Inheritance allows one class to acquire the properties and methods of another class.

Example

```
class Person {  
    public:  
        string name;  
};  
  
class Student : public Person {  
    public:  
        int rollNo;  
};
```

5. Polymorphism

Polymorphism means "many forms." It allows the same function or operator to behave differently in different situations.

Function Overloading Example

```
class Calculator {  
    public:  
        int add(int a, int b) {  
            return a + b;  
        }  
  
        double add(double a, double b) {  
            return a + b;  
        }  
};
```

6. Abstraction

Abstraction means hiding implementation details and showing only the essential features to the user.

Example

```
class Car {  
    public:  
        void startCar() {  
            cout << "Car Started";  
        }  
};
```

2. What are classes and objects in C++? Provide an example.

Answer:

Class

A **class** is a user-defined data type that acts as a **blueprint** for creating objects. It contains data members (variables) and member functions (methods) that define the properties and behavior of an object.

Object

An **object** is an instance of a class. It is used to access the data members and member functions defined in the class.

Ex:

```
#include <iostream>
```

```
using namespace std;
```

```
// Class Definition
```

```
class Student {
```

```
public:
```

```
    string name;
```

```
    int age;
```

```
    void display() {
```

```
        cout << "Name: " << name << endl;
```

```
        cout << "Age: " << age << endl;
```

```
    }
```

```
};
```

```
int main() {
```

```
    Student s1;
```

```
    s1.name = "Rahul";
```

```
s1.age = 20;

s1.display();

return 0;
}
```

3. What is inheritance in C++? Explain with an example.

Answer:

Inheritance is a feature of Object-Oriented Programming (OOP) in which a **new class (derived class)** acquires the properties and behaviors of an **existing class (base class)**.

```
#include <iostream>

using namespace std;
```

```
// Base Class

class Person {

public:

    string name;

    int age;


    void displayPerson() {

        cout << "Name: " << name << endl;

        cout << "Age: " << age << endl;

    }

};
```

```
// Derived Class

class Student : public Person {
```

```
public:

    int rollNo;

    void displayStudent() {

        displayPerson(); // inherited function

        cout << "Roll No: " << rollNo << endl;

    }

};
```

```
int main() {

    Student s1;

    s1.name = "Rahul";

    s1.age = 20;

    s1.rollNo = 101;

    s1.displayStudent();

    return 0;

}
```

4. What is encapsulation in C++? How is it achieved in classes?

Answer:

Encapsulation is one of the core concepts of Object-Oriented Programming (OOP). It means wrapping data (variables) and methods (functions) together into a single unit (class) and restricting direct access to some of the data.

In simple words:

Encapsulation = Data hiding + controlled access

Encapsulation is achieved using:

1. Classes

All data and functions are placed inside a class.

2. Access Specifiers

- private → Data cannot be accessed outside the class
- public → Data can be accessed from outside the class
- protected → Used in inheritance

Example:

```
#include <iostream>
using namespace std;

class BankAccount {
private:
    double balance; // hidden data (encapsulation)

public:
    // Constructor
    BankAccount() {
        balance = 0;
    }

    // Public method to deposit money
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    // Public method to withdraw money
    void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
        } else {
            cout << "Insufficient balance!" << endl;
        }
    }
}
```

```
// Public method to view balance
double getBalance() {
    return balance;
}
};

int main() {
    BankAccount account;

    account.deposit(1000);
    account.withdraw(300);

    cout << "Balance: " << account.getBalance();

    return 0;
}
```